

GPTQ & Optimal Brain Surgeon, Worked Out by Hand

QUANTIZE ONE COLUMN, THEN NUDGE THE REST TO ABSORB ITS ROUNDING ERROR

What this document does. It works a complete GPTQ pass on a layer small enough to do by hand — one output row, two correlated input channels — and shows the headline result: plain round-to-nearest leaves a reconstruction error of 0.5436, while GPTQ on the *same* integer grid leaves 0.0146, a 97.3% reduction. Nothing is asserted without arithmetic; every number is reproduced by Appendix A (`m02_t02_gptq_engine.py`). By the end you can compute a Hessian $H = X^\top X$, invert it, and run the Optimal Brain Surgeon compensation update with a pencil.

The one idea. When you round a weight, you do not just damage that weight — you damage the *layer's output*. But if a second, not-yet-quantized weight feeds a *correlated* input channel, you can deliberately mis-set it so its own output contribution *cancels* the first weight's rounding error. The Hessian $H = X^\top X$ tells you exactly how much to nudge. Rounding errors stop accumulating and start cancelling.

CONCEPTS COVERED, IN ORDER

the reconstruction objective $\min \|WX^\top - \hat{W}X^\top\|^2$; the input Hessian $H = X^\top X$ as a curvature proxy; the Optimal Brain Surgeon (1993) update; the GPTQ per-column compensation $\delta = (w_q - q(w_q))/[H^{-1}]_{qq} \cdot H^{-1}[:, q]$; a fully worked 2×2 case; why correlation is what makes it work; cost and limits.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The object under study	2
2	Step 1 — The reconstruction objective	2
3	Step 2 — Compute the Hessian and its inverse	3
4	Step 3 — The Optimal Brain Surgeon update	3
5	Step 4 — A complete pass by hand	3
5.1	Baseline: naive round-to-nearest	4
5.2	GPTQ: column 0, then compensate, then column 1	4
6	Step 5 — The full algorithm and its cost	4
7	The argument, at a glance	5
7.1	Equation cheat-sheet	5

Appendix A — Reference implementation

5

1 The object under study

A linear layer has weight matrix W of shape $[d_{\text{out}} \times d_{\text{in}}]$ and sees a batch of calibration activations X of shape $[N \times d_{\text{in}}]$. The forward pass is $Y = WX^\top$. We keep one output row and two input channels so W is 1×2 .

Quantity	Symbol	Value here
Weight row	W	$[0.70, 0.20]$
Calibration batch	X	4 samples $\times$ 2 channels (positively correlated)
Grid spacing	Δ	0.5 (deliberately coarse, so errors are visible)
Target output	$Y = WX^\top$	$[0.88, 0.70, 1.04, 0.79]$

$$X = \begin{bmatrix} 1.0 & 0.9 \\ 0.8 & 0.7 \\ 1.2 & 1.0 \\ 0.9 & 0.8 \end{bmatrix}.$$

The two columns rise and fall together — that correlation is the entire lever GPTQ pulls.

Intuition

Imagine two people carrying a table (the layer’s output). If person A (weight 0) suddenly lifts too high — a rounding error — the table tilts. GPTQ tells person B (weight 1), who is standing on the *same side*, to lift a little less, so the table stays level. The correlation between the two input channels is what puts B “on the same side” as A. With uncorrelated channels, B cannot help, and GPTQ degrades to plain rounding.

2 Step 1 — The reconstruction objective

We do not care about matching the weights; we care about matching the *output* the layer produces on real data. So the objective is output reconstruction error:

GPTQ objective — minimize reconstruction error

$$\hat{W}^* = \arg \min_{\hat{W}} \|WX^\top - \hat{W}X^\top\|_2^2 \quad \text{s.t.} \quad \hat{W} \text{ on the integer grid.}$$

Expand the squared norm. With $\Delta W = W - \hat{W}$,

$$\|\Delta W X^\top\|_2^2 = \text{trace}(\Delta W X^\top X \Delta W^\top) = \Delta W H \Delta W^\top, \quad H \equiv X^\top X.$$

The Hessian is the whole story

The error of a weight perturbation is a quadratic form in $H = X^\top X$. H is exactly the Hessian of the reconstruction loss with respect to W (the loss is quadratic, so its curvature is constant). It encodes which input dimensions carry energy and how they correlate — precisely the information needed to decide how to repair a rounding error.

This is the Optimal Brain Surgeon insight (Hassibi & Stork, 1993) carried from pruning to quantization.

3 Step 2 — Compute the Hessian and its inverse

$$H = X^\top X \text{ and } H^{-1}$$

$$H = X^\top X = \begin{bmatrix} 3.89 & 3.38 \\ 3.38 & 2.94 \end{bmatrix}, \quad H^{-1} = \begin{bmatrix} 240.98 & -277.05 \\ -277.05 & 318.85 \end{bmatrix}.$$

The diagonal $[H^{-1}]_{qq} = (240.98, 318.85)$ measures how “isolated” each weight is: a large value means the channel is highly correlated with the others, so its error is easy to absorb elsewhere. The big off-diagonal (-277.05) is the coupling GPTQ exploits. ✓

Watch out

Real transformer Hessians are $d_{\text{in}} \times d_{\text{in}}$ (e.g. 8192^2) and badly conditioned — condition numbers above 10^8 are routine. Inverting them directly produces NaN-filled weights. Production GPTQ adds a small damping λI (about 1% of the mean diagonal) and uses a Cholesky factorization $H^{-1} = LL^\top$ instead of an explicit inverse. Skipping the damping is the #1 reason a homegrown GPTQ blows up.

4 Step 3 — The Optimal Brain Surgeon update

OBS asks: if I force weight q onto the grid (a perturbation $\delta w_q = \hat{w}_q - w_q$), what is the *cheapest* way to adjust the remaining free weights to keep the output unchanged? The closed-form answer, minimizing $\Delta W H \Delta W^\top$ subject to fixing weight q , is:

GPTQ / OBS per-column compensation

$$\delta = \frac{w_q - q(w_q)}{[H^{-1}]_{qq}}, \quad W[\text{cols} > q] \leftarrow \delta \cdot H^{-1}[q, \text{cols} > q].$$

In words: take the rounding residual $w_q - q(w_q)$, scale it by the inverse-Hessian diagonal entry, and spread that across the still-unquantized columns weighted by the inverse-Hessian *row*. The columns already quantized are frozen; only future columns absorb the correction.

Intuition

The residual is “debt” the layer’s output owes after you rounded weight q . $[H^{-1}]_{qq}$ converts that weight-space debt into a normalized correction; the row $H^{-1}[q, > q]$ says how to distribute it across the unquantized weights so the output ledger balances. A weight strongly coupled to q (large H^{-1} entry) takes on more of the debt.

5 Step 4 — A complete pass by hand

5.1 Baseline: naive round-to-nearest

Snap each weight independently to the grid of spacing 0.5:

$$W = [0.70, 0.20] \xrightarrow{\text{round}/\Delta} \hat{W}_{\text{RTN}} = [0.50, 0.00].$$

The reconstruction error is

$$\|(W - \hat{W}_{\text{RTN}})X^\top\|^2 = 0.5436.$$

Both weights rounded *down*, and because both channels are positive and correlated, the two errors *add* rather than cancel — the worst case.

5.2 GPTQ: column 0, then compensate, then column 1

Column 0

$$w_0 = 0.70 \xrightarrow{\text{round}} q(w_0) = 0.50, \quad \text{residual} = 0.70 - 0.50 = +0.20.$$

$$\delta = \frac{0.20}{[H^{-1}]_{00}} = \frac{0.20}{240.98} = +0.00083.$$

$$W[1] \leftarrow \delta \cdot H^{-1}[0, 1] = 0.00083 \times (-277.05) = -0.2299.$$

$$\text{so } W[1] : 0.20 - (-0.2299) = +0.4299.$$

GPTQ *raised* weight 1 from 0.20 to 0.4299 *before* quantizing it.

Column 1

Now quantize the updated weight 1:

$$w_1 = 0.4299 \xrightarrow{\text{round}} q(w_1) = 0.50.$$

Because GPTQ pushed it up to 0.43, it rounds *up* to 0.5 — whereas the original 0.20 would have rounded *down* to 0. That flipped code is the whole trick.

$$\hat{W}_{\text{GPTQ}} = [0.50, 0.50], \quad \|(W - \hat{W}_{\text{GPTQ}})X^\top\|^2 = 0.0146. \quad \checkmark$$

The payoff

Same grid, same two integer codes available — but GPTQ chose $[0.5, 0.5]$ where RTN chose $[0.5, 0]$. Weight 0 rounded down (output too low); GPTQ let weight 1 round *up* (output too high) so the two nearly cancel on the correlated channels. Reconstruction error fell from 0.5436 to 0.0146 — a 97.3% **reduction**, with zero extra storage.

Mini-example — why correlation is mandatory

If the two channels were *uncorrelated*, H would be diagonal, H^{-1} would be diagonal, the off-diagonal $H^{-1}[0, 1]$ would be 0, and the compensation $\delta \cdot H^{-1}[0, 1]$ would vanish. GPTQ would do nothing beyond RTN. It is the off-diagonal coupling — the fact that real activations are correlated — that gives GPTQ something to work with. Dead (zero-energy) channels are detected as zero diagonals of H and handled separately.

6 Step 5 — The full algorithm and its cost

Scaled to a real layer, GPTQ is one Hessian build plus a left-to-right sweep:

#	Operation
1	$H = 2X^\top X + \lambda I$ (once per layer; damping $\lambda \approx 1\%$ of mean diag)
2	Cholesky $H^{-1} = LL^\top$ (stable access to the inverse)
3	For each column j : (a) $q_j = \text{quant}(W[:,j])$; (b) $\text{err} = (W[:,j] - q_j)/[H^{-1}]_{jj}$; (c) $W[:,j+1:] -= \text{err} \otimes H^{-1}[j, j+1:]$; (d) freeze $W[:,j] = q_j$.
4	Return the quantized matrix. Cost $\approx O(d^3)$ per layer.

On Llama-3-70B at W4, this turns RTN's -3.4 MMLU points into GPTQ's -0.5 :

Method	MMLU	Δ from FP16
FP16 baseline	79.5	—
RTN W4	76.1	-3.4
GPTQ W4	79.0	-0.5

Calibration needs only 128–1024 samples and 1–3 hours on a single H100, even for 70B.

Watch out

GPTQ quantizes **weights only** (W4A16): it assumes FP16 activations. You get the 4× memory and bandwidth win, but the matmul still runs in FP16 with dequantized weights — no extra *compute* speedup from INT8/FP8 tensor cores. For activation quantization (W8A8, W4A8) you need SmoothQuant or AWQ, which are Topics 3–4.

7 The argument, at a glance

#	Step	Result
1	Objective	$\min \ WX^\top - \hat{W}X^\top\ ^2 = \Delta W H \Delta W^\top$
2	Hessian	$H = X^\top X$; $[H^{-1}]_{00}=240.98$, $H^{-1}[0,1]=-277.05$
3	OBS update	$\delta = (w_q - q)/[H^{-1}]_{qq}$, spread by $H^{-1}[q, >q]$
4	Worked case	RTN $[0.5, 0] \rightarrow$ GPTQ $[0.5, 0.5]$; error $0.544 \rightarrow 0.015$
5	At scale	Llama-3-70B W4: RTN -3.4 vs GPTQ -0.5 MMLU

7.1 Equation cheat-sheet

Objective	$\hat{W}^* = \arg \min_{\hat{W}} \ WX^\top - \hat{W}X^\top\ _2^2$
Quadratic form	$\ \Delta W X^\top\ ^2 = \Delta W H \Delta W^\top$
Hessian	$H = X^\top X$ (damped: $2X^\top X + \lambda I$)
OBS scale	$\delta = (w_q - q)/[H^{-1}]_{qq}$
Compensation	$W[>q] -= \delta \cdot H^{-1}[q, >q]$
Per-column cost	controls $(w_q - q(w_q))^2/[H^{-1}]_{qq}$

Appendix A — Reference implementation

Every number above is produced by `m02_t02_gptq_engine.py`. Run it to reproduce the Hessian, the compensation, and the 97.3% error reduction; change W , X , or Δ to explore other cases.

```
1 """
2 Reference implementation for: Module 02 - Topic 2
3 "GPTQ / Optimal Brain Surgeon - quantize one column, compensate the rest."
```

```

4
5 We take a TINY linear layer (1 output row, 2 input columns) with a batch of
6 calibration activations X, and compare:
7   (a) naive round-to-nearest (RTN), and
8   (b) GPTQ with OBS-style error compensation
9 on the reconstruction error  $\|W X^T - \hat{W} X^T\|^2$ .
10
11 The two input channels are CORRELATED (the usual case in transformers): when
12 channel 0 is large, channel 1 tends to be large too. That correlation is what
13 lets GPTQ absorb column 0's rounding error by nudging column 1.
14
15 Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
16 """
17
18 import numpy as np
19
20 np.set_printoptions(precision=4, suppress=True)
21
22 # -----
23 # 0. The tiny layer. W is one row (1 x 2). Coarse grid (step = 0.5) so the
24 #    rounding error is large and the compensation is visible by hand.
25 # -----
26 W = np.array([0.70, 0.20])          # one output row, 2 input weights
27 STEP = 0.5                          # quantization grid spacing
28
29
30 def quant(x, step=STEP):
31     return np.round(x / step) * step
32
33
34 # Calibration activations: N=4 samples, 2 channels, positively correlated.
35 X = np.array([
36     [1.0, 0.9],
37     [0.8, 0.7],
38     [1.2, 1.0],
39     [0.9, 0.8],
40 ])
41 N = X.shape[0]
42 print("== 0. Tiny layer ==")
43 print("W =", W, "    grid step =", STEP)
44 print("X (4x2) =\n", X)
45
46 # -----
47 # 1. Hessian H =  $X^T X$  (2x2) and its inverse.
48 # -----
49 H = X.T @ X
50 Hinv = np.linalg.inv(H)
51 print("\n== 1. Hessian H =  $X^T X$  ==")
52 print("H =\n", H)
53 print("Hinv =\n", Hinv)
54 print("diag(Hinv) =", np.diag(Hinv))
55
56 Y = W @ X.T
57 print("\ntarget output Y =  $W X^T$  =", Y)
58
59
60 def recon_error(W_hat):
61     e = (W - W_hat) @ X.T
62     return float(e @ e)
63
64
65 # -----
66 # 2. NAIVE round-to-nearest.

```

```

67 # -----
68 W_rtn = quant(W)
69 print("\n== 2. Naive RTN ==")
70 print("W_rtn =", W_rtn)
71 print(f"reconstruction error ||(W-W_rtn)X^T||^2 = {recon_error(W_rtn):.6f}")
72
73 # -----
74 # 3. GPTQ: quantize column 0, push its error into column 1 via the
75 #   OBS update   W[1:] -= (err / Hinv[0,0]) * Hinv[0, 1:].
76 # -----
77 print("\n== 3. GPTQ with error compensation ==")
78 Wc = W.astype(float).copy()
79 Q = np.zeros_like(Wc)
80 d = W.size
81 for q in range(d):
82     w_q = Wc[q]
83     q_q = quant(w_q)
84     Q[q] = q_q
85     err = w_q - q_q
86     delta = err / Hinv[q, q]
87     print(f"   col {q}: w={w_q:+.4f} -> q={q_q:+.4f}   resid={err:+.4f}   "
88           f"Hinv[{q},{q}]={Hinv[q,q]:.4f}   delta={delta:+.4f}")
89     if q + 1 < d:
90         update = delta * Hinv[q, q + 1:]
91         Wc[q + 1:] -= update
92         print(f"       compensate col {q+1}: W[{q+1}] {Wc[q+1]+update[0]:+.4f}   "
93               f"   - ({update[0]:+.4f}) = {Wc[q+1]:+.4f}")
94
95 print("\nGPTQ quantized weights W_hat =", Q)
96 print(f"reconstruction error ||(W-W_hat)X^T||^2 = {recon_error(Q):.6f}")
97
98 # -----
99 # 4. Verdict.
100 # -----
101 e_rtn = recon_error(W_rtn)
102 e_gptq = recon_error(Q)
103 print("\n== 4. Verdict ==")
104 print(f"   RTN   error = {e_rtn:.6f}")
105 print(f"   GPTQ error = {e_gptq:.6f}")
106 print(f"   GPTQ reduces reconstruction error by {(1-e_gptq/e_rtn)*100:.1f}%")
107 print("   (both snap to the SAME grid; only the compensation differs)")
108
109 # -----
110 # 5. Why it works: column 1 was nudged from 0.20 -> its new value so that
111 #   its OWN rounding now partly cancels column 0's leftover error.
112 # -----
113 print("\n== 5. Intuition check ==")
114 print(f"   col0 left residual {W[0]-quant(W[0]):+.4f} on a channel correlated with col1.")
115 print(f"   GPTQ moved col1 to {Wc[1]+0:+.4f} before its own rounding, steering the")
116 print(f"   final col1 code to {Q[1]:+.4f} so the output error partly cancels.")

```

— END OF DOCUMENT —